JAVA PROGRAMMING — PART 2

Continuation of Part 1: methods, strings, inheritance, interfaces, and packages

Page 1 — Methods in More Detail

A method is a named block of code that performs a task. Methods reduce repetition and break a large problem into smaller parts. A method can accept parameters and may return a value.

```
static int add(int a, int b) {
    return a + b;
}

public static void main(String[] args) {
    int result = add(4, 6);
    System.out.println(result); // 10
}
```

Parameters are variables in the method definition; arguments are the actual values passed during a call. The keyword void means a method does not return a value.

Method overloading

Overloading uses the same method name with different parameter lists. The number, types, or order of parameters must differ; changing only the return type is not enough.

```
static int multiply(int a, int b) { return a * b; }
static double multiply(double a, double b) { return a * b; }
```

Page 2 — Strings and Keyboard Input

A String represents text and is immutable: operations produce new strings rather than changing the original object.

```
String name = "Java";
System.out.println(name.length());       // 4
System.out.println(name.toUpperCase());  // JAVA
System.out.println(name.charAt(0));      // J
System.out.println(name.contains("av")); // true
```

Use equals() to compare String contents. == checks whether references point to the same object, so it is usually not suitable for comparing text values.

```
String a = new String("hello");
String b = new String("hello");
System.out.println(a.equals(b)); // true
System.out.println(a == b);      // false in this example
```

Reading keyboard input

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Enter age: ");
int age = sc.nextInt();
System.out.println("Age: " + age);
sc.close();
```

Scanner includes nextInt(), nextDouble(), and nextLine(). When mixing nextInt() and nextLine(), remember that nextInt() leaves the line break behind; consume it if needed before reading a full line.

Page 3 — Inheritance and Polymorphism

Inheritance lets a child class reuse or specialize accessible members of a parent class. Java uses extends for class inheritance. A child can call a parent constructor with super(...), and can override an inherited instance method.

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
Animal pet = new Dog();
pet.sound(); // Bark
```

This is also runtime polymorphism: a parent-type reference points to a child object, and the overridden method is selected according to the actual object.

Important keywords

- **extends:** declares class inheritance.
- **super:** refers to parent members or invokes a parent constructor.
- **this:** refers to the current object.
- **final:** can prevent reassignment, overriding, or inheritance depending on where it is used.

Page 4 — Abstract Classes and Interfaces

An abstract class cannot be instantiated directly and can contain both implemented and abstract methods. An interface defines a contract that implementing classes follow; a class can implement multiple interfaces.

```
interface Payable {
    double getPayment();
}
class Invoice implements Payable {
    private double amount;
    Invoice(double amount) { this.amount = amount; }
    public double getPayment() { return amount; }
}
```

Use an interface when different classes should share a capability. Use an abstract class when related classes need shared state or implementation. Modern interfaces can also contain default and static methods.

Access modifiers

- **public:** broadly accessible subject to Java and module rules.
- **private:** accessible only inside the declaring class.
- **protected:** accessible in the same package and through inheritance under Java's access rules.
- **Package-private:** no modifier; generally accessible only within the same package.

Page 5 — Packages, Static Members, and Practice

Packages group related classes and help avoid naming conflicts. A package declaration appears near the beginning of a source file; import statements allow types to be referred to by simple names.

```
package college.records;
import java.util.ArrayList;

public class Demo {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Aman");
        System.out.println(names);
    }
}
```

The static keyword marks a member that belongs to the class rather than to an individual object. A static method cannot directly access instance fields without an object reference.

```
class Counter {
    static int total = 0;
    Counter() { total++; }
}
new Counter();
new Counter();
System.out.println(Counter.total); // 2
```

Practice questions

- Overload area() to calculate a square and rectangle's area.
- Read a sentence and print its length.
- Create a Person class and a Student subclass with a roll number.
- Create a Printable interface and implement it in two classes.
- Explain overloading versus overriding with code.

Part 2 recap

Practise methods, Strings, keyboard input, inheritance, polymorphism, interfaces, access control, packages, and static members before moving to exceptions, collections, generics, file handling, and threads.